```
================================================================================
   DISTRIBUTED TICKET BOOKING SYSTEM — 40-DAY COMPLETE ROADMAP
   Node.js | Express | MongoDB | Redis | gRPC | Microservices
   For: Bhupesh | Goal: Machine Coding Round + System Design Interview
================================================================================

"Build it like a senior engineer. Understand every decision before you write
 every line of code. That is what separates average devs from system engineers."

================================================================================
PAGE 1 — MASTER OVERVIEW
================================================================================

WHAT ARE YOU BUILDING?
----------------------
A high-concurrency distributed backend system where multiple users try to book
limited seats simultaneously. The system must guarantee no double booking,
handle
payment failures gracefully, and scale to handle traffic spikes.

Real-world systems using this exact architecture:
  - BookMyShow
  - IRCTC
  - Airline booking (MakeMyTrip, Goibibo)
  - Event ticketing (Ticketmaster, Insider)

Why this project is powerful for interviews:
  - Covers concurrency (top interview topic)
  - Covers distributed systems (rare skill among candidates)
  - Has clear HLD + LLD depth
  - Not a CRUD app — has real engineering problems to solve


6-WEEK JOURNEY AT A GLANCE
---------------------------
  Week 1 (Days 1–7)   → Foundation + Auth
  Week 2 (Days 8–14)  → Concurrency + DB Locking
  Week 3 (Days 15–20) → Redis + Distributed Locking
  Week 4 (Days 21–26) → Payment + Idempotency + Saga
  Week 5 (Days 27–32) → Queue + Microservices + gRPC
  Week 6 (Days 33–40) → Production Readiness + Interview Polish


ARCHITECTURE EVOLUTION (Simple → Scalable)
------------------------------------------
  Phase 1 (Weeks 1–2):  Monolith — Client → Express Server → MongoDB
                         Learn: auth, schema, race condition, DB locking

  Phase 2 (Weeks 3–4):  Add Redis + Queue
                         Client → Server → Redis (locks) + MongoDB + BullMQ

  Phase 3 (Weeks 5–6):  Microservices
                         Client → API Gateway → Booking Service ←gRPC→ Payment
Service
                                                          ↓ BullMQ
                                                  Notification Service
                         All services share MongoDB + Redis

GOLDEN RULE: Do NOT jump to microservices on Day 1.
Build monolith first. Understand problems. Then split. That is how real
systems evolve and that is what interviewers want to hear.


CORE BOOKING FLOW (What You Are Building)
```

```
-----------------------------------------
  1. User Request     → authenticated API call
  2. Auth Check       → verify JWT, check permissions
  3. Fetch Seats      → from cache (Redis) first, DB on miss
  4. Lock Seat        → atomic Redis SETNX with 5-min TTL
  5. Payment Gateway  → initiate Razorpay/Stripe order
  6. Confirm Booking  → webhook received, update state
  7. Send Ticket      → async via notification queue

Failure paths (equally important):
  - Payment fails     → release seat lock → update state to FAILED → notify user
  - Lock expires      → cron job releases seat → user must retry
  - Server crash      → Redis TTL auto-releases lock → queue retries job


BOOKING STATE MACHINE (Must Know Cold)
---------------------------------------
  INIT → LOCKED → PAYMENT_PENDING → CONFIRMED
                        ↓
                     FAILED → (seat released back to AVAILABLE)
                        ↓
                    EXPIRED   → (TTL expired, seat released)

Rules:
  - All state transitions must be atomic
  - No booking can skip a state (INIT → CONFIRMED is invalid)
  - FAILED and EXPIRED both release the seat back to AVAILABLE
  - Store every state change with timestamp for audit trail


DAILY 1–1.5 HOUR ROUTINE
--------------------------
  First 15 min  → RESEARCH
                  Before coding, ask: what problem am I solving?
                  What solutions exist? What are the trade-offs?
                  Draw the problem on paper first.

  Next 40 min   → CODE
                  Build the feature. Run it. Test it manually.
                  Use curl or Postman to verify.

  Last 15 min   → NOTES
                  Write down: what did I build, what decision did I make,
                  what trade-off did I choose and why.
                  This becomes your interview answer.


================================================================================
PAGE 2 — WEEK 1: FOUNDATION + AUTH (Days 1–7)
================================================================================

GOAL: Working API with auth, clean schema, booking can be created (no locking
yet)

-------------------------------------------------------------------------
Day | Topic                | What to build / Research first
-------------------------------------------------------------------------
 1  | Project setup        | Folder structure (see below), Express server,
    |                      | Mongoose connect, dotenv config, git init
    |                      | Research: MVC vs modular folder structure
-------------------------------------------------------------------------
 2  | User auth (part 1)   | POST /auth/register, POST /auth/login
    |                      | bcrypt password hash, JWT access token (15min)
    |                      | Research: why bcrypt? what is salt rounds?
```

3	User auth (part 2)	Refresh token (7 days), POST /auth/logout, auth middleware, role guard (user/admin) Research: how to store refresh token securely?
4	Schema design	User, Show, Venue, Seat, Booking models Think carefully: what fields does Seat need? Research: embed vs reference in MongoDB
5	Seat + Show API	GET /shows, GET /shows/:id/seats Seat status enum: AVAILABLE / LOCKED / BOOKED Research: MongoDB indexing for showId queries
6	Booking API (basic)	POST /bookings — create booking, state = INIT No locking yet, just store the record Research: what fields must a booking have?
7	Error handling + Postman tests	Global error handler middleware Input validation with Joi or Zod Write 5 test flows in Postman collection

MILESTONE: Login works. Can fetch seats. Can create a booking record.
No concurrency handled yet — that is Week 2.

FOLDER STRUCTURE (Use This Exactly)

```
src/
├── modules/
│   ├── auth/
│   │   ├── auth.controller.js
│   │   ├── auth.service.js
│   │   ├── auth.repository.js
│   │   └── auth.routes.js
│   ├── booking/
│   │   ├── booking.controller.js
│   │   ├── booking.service.js
│   │   ├── booking.repository.js
│   │   └── booking.routes.js
│   └── seat/
│       ├── seat.controller.js
│       ├── seat.service.js
│       ├── seat.repository.js
│       └── seat.routes.js
├── middleware/
│   ├── auth.middleware.js
│   ├── error.middleware.js
│   └── validate.middleware.js
├── config/
│   ├── db.js
│   └── redis.js
├── utils/
│   ├── jwt.js
│   └── hash.js
└── app.js
```

MONGODB SCHEMAS (Key Fields)

User:

```
{ _id, email, passwordHash, role: ['user','admin'], createdAt }
```

Venue:

```
  { _id, name, city, totalSeats }

Show:
  { _id, venueId, movieName, date, time, totalSeats }

Seat:
  { _id, showId, row, col, status: ['AVAILABLE','LOCKED','BOOKED'],
    version: Number,       ← for optimistic locking
    lockedBy: userId,      ← who locked it
    lockedAt: Date }       ← when locked (for TTL cron)

Booking:
  { _id, userId, showId, seatIds: [],
    state: ['INIT','LOCKED','PAYMENT_PENDING','CONFIRMED','FAILED','EXPIRED'],
    idempotencyKey: String,
    paymentId: String,
    amount: Number,
    createdAt, updatedAt }


JWT AUTH FLOW
--------------
  Register → hash password → save user → return tokens
  Login    → verify password → generate accessToken (15min) + refreshToken (7d)
           → store refreshToken in DB (or Redis blacklist)
  Request  → Authorization: Bearer <accessToken>
           → middleware verifies → attaches user to req.user
  Logout   → blacklist refreshToken in Redis
  Refresh  → POST /auth/refresh with refreshToken → new accessToken


KEY CONCEPTS WEEK 1
--------------------
  JWT:
    - Access token: short-lived (15 min), stateless verification
    - Refresh token: long-lived (7 days), stored server-side
    - Never store access token in localStorage (XSS risk), use httpOnly cookie
    - Blacklist on logout by storing token ID in Redis with TTL

  Input validation:
    - Always validate before hitting service layer
    - Use Joi or Zod schema — never trust client input
    - Return 400 with clear error message

  MongoDB indexing:
    - Index showId on Seat for fast seat lookups
    - Compound index: { showId: 1, status: 1 } for available seats query
    - Index userId on Booking for user's booking history


================================================================================
PAGE 3 — WEEK 2: CONCURRENCY + DB LOCKING (Days 8–14)
================================================================================

GOAL: 2 users click same seat simultaneously → exactly 1 succeeds, other gets
clean error. Zero double bookings. This is the most important week.

-------------------------------------------------------------------------
Day | Topic                | What to build / Research first
-------------------------------------------------------------------------
 8  | Race condition       | Research: draw the timeline of double booking
    | deep dive            | Thread A reads available, Thread B reads available,
    |                      | both write BOOKED. Understand WHY this happens.
-------------------------------------------------------------------------
```

```
 9  | Optimistic locking   | Add 'version' field to Seat model (default: 0)
    |                      | findOneAndUpdate with { version: currentVersion }
    |                      | If version mismatch → throw ConflictError → retry
-------------------------------------------------------------------------
10  | Pessimistic locking  | Use atomic findOneAndUpdate with condition:
    |                      | { _id: seatId, status: 'AVAILABLE' }
    |                      | { $set: { status: 'LOCKED', lockedBy: userId } }
    |                      | Compare: which is faster? When to use each?
-------------------------------------------------------------------------
11  | Mongo transactions   | Multi-document transaction using session:
    |                      | Lock seat + create booking in single atomic op
    |                      | If booking creation fails, seat lock rolls back
-------------------------------------------------------------------------
12  | Isolation levels     | Research: Read Uncommitted, Read Committed,
    |                      | Repeatable Read, Serializable — with examples
    |                      | What does MongoDB use by default?
-------------------------------------------------------------------------
13  | Wire everything      | Full flow: lock seat → create LOCKED booking
    |                      | Handle 409 Conflict response properly
    |                      | Add retry logic (max 3 retries with backoff)
-------------------------------------------------------------------------
14  | Stress test          | Simulate concurrent requests:
    |                      | Promise.all(Array(10).fill(bookSeat(seatId)))
    |                      | Verify: exactly 1 success, 9 conflicts
-------------------------------------------------------------------------

MILESTONE: Run 10 concurrent requests on same seat. Exactly 1 succeeds.
Check DB — seat is LOCKED once, not 10 times. No data corruption.


THE RACE CONDITION PROBLEM (Draw This First)
--------------------------------------------
  Timeline without protection:
    t=0  Request A: SELECT seat WHERE id=1 → status=AVAILABLE ✓
    t=1  Request B: SELECT seat WHERE id=1 → status=AVAILABLE ✓
    t=2  Request A: UPDATE seat SET status=LOCKED WHERE id=1  ← both succeed!
    t=3  Request B: UPDATE seat SET status=LOCKED WHERE id=1  ← DOUBLE BOOKING!

  Why it happens:
    - Read and write are two separate operations
    - Between read and write, another request can read the same data
    - This is called TOCTOU: Time Of Check vs Time Of Use

  Solution: Make check-and-set ATOMIC (one operation, no gap)
    findOneAndUpdate(
      { _id: seatId, status: 'AVAILABLE' },   ← check
      { $set: { status: 'LOCKED' } },          ← set
      { new: true }
    )
    If result is null → seat was already taken → throw 409 Conflict


OPTIMISTIC LOCKING (Version Field)
----------------------------------
  How it works:
    1. Read seat with version: { _id: 1, status: 'AVAILABLE', version: 5 }
    2. Attempt update:
       findOneAndUpdate(
         { _id: seatId, version: 5 },           ← must match
         { $set: { status: 'LOCKED' }, $inc: { version: 1 } }
       )
    3. If another request updated first → version is now 6 → query returns null
    4. Retry up to 3 times with exponential backoff (100ms, 200ms, 400ms)
```

```
  When to use: Low write contention. Many reads, occasional writes.
  Trade-off: Retries add latency under contention.


PESSIMISTIC LOCKING (Atomic Condition)
---------------------------------------
  How it works:
    findOneAndUpdate(
      { _id: seatId, status: 'AVAILABLE' },    ← check AND lock in one query
      { $set: { status: 'LOCKED', lockedBy: userId, lockedAt: new Date() } },
      { new: true }
    )
    If result is null → already locked → throw 409 immediately (no retry)

  When to use: High write contention. Many concurrent writers on same seat.
  Trade-off: No retry needed, but slightly more DB load for popular seats.


LOCKING COMPARISON TABLE
-------------------------
  Feature              | Optimistic Lock      | Pessimistic Lock
  ---------------------|----------------------|----------------------
  Mechanism            | Version field check  | Atomic condition query
  On conflict          | Retry (up to 3x)     | Fail immediately (409)
  Best for             | Low contention       | High contention
  DB load              | Lower                | Slightly higher
  Deadlock risk        | None                 | None (MongoDB)
  Code complexity      | Medium               | Low


MONGODB TRANSACTION EXAMPLE
----------------------------
  const session = await mongoose.startSession();
  session.startTransaction();
  try {
    const seat = await Seat.findOneAndUpdate(
      { _id: seatId, status: 'AVAILABLE' },
      { $set: { status: 'LOCKED', lockedBy: userId } },
      { session, new: true }
    );
    if (!seat) throw new Error('Seat not available');

    const booking = await Booking.create([{
      userId, seatIds: [seatId], state: 'LOCKED'
    }], { session });

    await session.commitTransaction();
    return booking[0];
  } catch (err) {
    await session.abortTransaction();
    throw err;
  } finally {
    session.endSession();
  }


ISOLATION LEVELS (Know for Interview)
--------------------------------------
  Read Uncommitted: Can read data not yet committed (dirty read). Never use.
  Read Committed:   Only reads committed data. Default in most DBs.
  Repeatable Read:  Same query returns same result within transaction.
  Serializable:     Complete isolation. Transactions run as if sequential.
                    Safest but slowest. Use for financial operations.
```

MongoDB default: Snapshot isolation (similar to Repeatable Read).
For transactions: Can configure readConcern/writeConcern.

==
PAGE 4 — WEEK 3: REDIS + DISTRIBUTED LOCKING (Days 15–20)
==

GOAL: Seat locks stored in Redis with TTL. Auto-release on timeout.
Seat availability served from cache. DB load reduced significantly.

Day	Topic	What to build / Research first
15	Redis fundamentals	Research: String, Hash, List, Set, Sorted Set Install Redis locally. Try SET, GET, EXPIRE, TTL commands in redis-cli. Understand in-memory.
16	SETNX lock	SET seat:{seatId} {userId} NX PX 300000 NX = only if Not eXists, PX = milliseconds TTL Implement lockSeat() and releaseSeat() functions
17	Lock release + renew	DEL seat:{seatId} on payment confirm/fail Implement keepalive: extend TTL if user active Research: compare-and-delete (Lua script)
18	Redlock algorithm	Research: why single-node Redis lock is not safe What if Redis restarts? Lock is lost. Implement basic Redlock with 3 Redis instances
19	Cache seat availability	Cache-aside pattern for seat availability: Check Redis → miss → query DB → populate Redis Key: show:{showId}:seats, TTL: 60 seconds
20	Cron cleanup	BullMQ cron job: every 60 seconds Find seats where lockedAt < now - 5min Set status back to AVAILABLE, clear lockedBy

MILESTONE: Lock a seat → wait 5 min → seat auto-released. Cache reduces
DB queries by 80% for seat availability. Redlock prevents split-brain.

WHY REDIS OVER DB LOCK? (Interview Answer)

Redis:
+ In-memory: O(1) operations, microsecond response
+ TTL built-in: auto-release on crash, no zombie locks
+ SETNX is atomic: single command, no race condition
+ Cross-service: works across multiple Node.js instances
+ Built for this: designed for distributed coordination

MongoDB:
- Disk-based: slower for lock checks
- No native TTL for lock documents (need cron cleanup)
- Higher load: every seat check hits DB
- Connection overhead: transactions use sessions

Choose Redis for locks. Keep MongoDB for persistent booking data.

REDIS KEY DESIGN

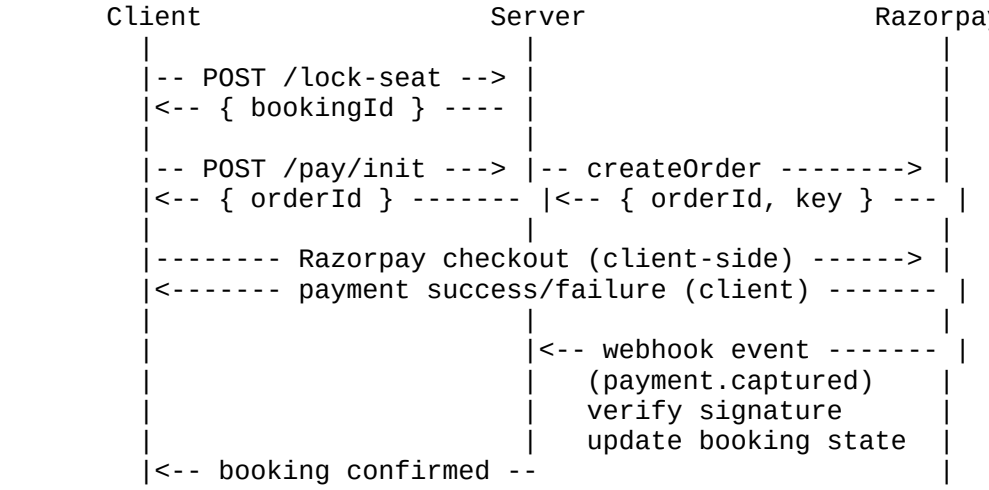
Key pattern	Type	Value	TTL	Purpose

```
-------------------------|--------|------------------|--------|----------------
-
  seat:lock:{seatId}       | STRING | userId           | 300s   | Distributed
lock
  show:{showId}:seats      | HASH   | seatId → status  | 60s    | Availability
cache
  idem:{hash}              | STRING | response JSON    | 86400s | Idempotency
store
  booking:session:{bookId} | HASH   | booking object   | 600s   | Session state
  ratelimit:{userId}:{min} | STRING | request count    | 60s    | Rate limiting
  refresh:{userId}         | STRING | token hash       | 604800s| Refresh token


REDIS LOCK IMPLEMENTATION
--------------------------
  // Lock a seat
  async function lockSeat(seatId, userId, ttlMs = 300000) {
    const key = `seat:lock:${seatId}`;
    const result = await redis.set(key, userId, 'NX', 'PX', ttlMs);
    return result === 'OK'; // true = locked, false = already taken
  }

  // Release only if we own the lock (Lua script for atomicity)
  const releaseScript = `
    if redis.call('get', KEYS[1]) == ARGV[1] then
      return redis.call('del', KEYS[1])
    else
      return 0
    end
  `;
  async function releaseSeat(seatId, userId) {
    const key = `seat:lock:${seatId}`;
    return redis.eval(releaseScript, 1, key, userId);
  }

  // Extend lock TTL (user still on payment page)
  async function extendLock(seatId, userId, ttlMs = 300000) {
    const key = `seat:lock:${seatId}`;
    const owner = await redis.get(key);
    if (owner !== userId) throw new Error('Lock not owned by you');
    await redis.pexpire(key, ttlMs);
  }


REDLOCK ALGORITHM (Why It Matters)
-----------------------------------
  Problem with single Redis node:
    - Redis restarts → all locks lost → multiple nodes acquire same lock
    - Network partition → client thinks lock acquired, Redis disagrees

  Redlock solution:
    - Use N Redis instances (usually 3 or 5)
    - Acquire lock on majority (N/2 + 1) nodes within time limit
    - Lock valid only if acquired on majority AND total time < TTL
    - Release on ALL nodes

  For this project: Use node-redlock npm package.
  Interview answer: "For production, I use Redlock across 3 Redis nodes to
  handle single node failures. For this project scale, single node Redis
  with TTL is acceptable as a known trade-off."


CACHE-ASIDE PATTERN
```

```
--------------------
  async function getSeatsForShow(showId) {
    const cacheKey = `show:${showId}:seats`;

    // 1. Check cache
    const cached = await redis.get(cacheKey);
    if (cached) return JSON.parse(cached);

    // 2. Cache miss → query DB
    const seats = await Seat.find({ showId });

    // 3. Populate cache with TTL
    await redis.setex(cacheKey, 60, JSON.stringify(seats));

    return seats;
  }

  // Cache invalidation: when a seat is locked/booked
  async function invalidateSeatCache(showId) {
    await redis.del(`show:${showId}:seats`);
  }

  Interview note: Always invalidate cache when data changes.
  Stale cache = user sees available seat that is actually booked.


================================================================================
PAGE 5 — WEEK 4: PAYMENT + IDEMPOTENCY + SAGA (Days 21–26)
================================================================================

GOAL: Payment fail is fully handled. Double-tap pay is impossible.
Entire money flow is safe and recoverable.

-------------------------------------------------------------------------
Day | Topic                | What to build / Research first
-------------------------------------------------------------------------
21  | Payment gateway      | Research Razorpay order flow: create order →
    | setup                | get order_id → client pays → webhook fires
    |                      | Set up test mode, get API keys, read webhook docs
-------------------------------------------------------------------------
22  | Payment initiation   | POST /payment/initiate
    |                      | → create order in Razorpay
    |                      | → update booking state to PAYMENT_PENDING
    |                      | → return { orderId, amount, currency } to client
-------------------------------------------------------------------------
23  | Webhook handler      | POST /webhook/payment (public, no auth middleware)
    |                      | Verify HMAC-SHA256 signature
    |                      | payment.captured → confirm booking
    |                      | payment.failed → release seat, update state
-------------------------------------------------------------------------
24  | Idempotency keys     | Generate key: sha256(userId + seatId + amount)
    |                      | Store in Redis: idem:{key} → response (TTL 24h)
    |                      | On duplicate request: return stored response
-------------------------------------------------------------------------
25  | Saga pattern         | Implement compensation flow:
    |                      | payment.failed → DEL seat lock → FAILED state
    |                      | → notify user → optionally trigger refund
-------------------------------------------------------------------------
26  | End-to-end test      | Full happy path + all failure paths:
    |                      | lock → pay → webhook(success) → CONFIRMED
    |                      | lock → pay → webhook(failed) → seat released
    |                      | lock → pay → pay again → idempotent response
-------------------------------------------------------------------------
```

MILESTONE: Pay fails → seat released, no charge. Tap pay 3 times →
only 1 charge, others get cached response. Complete audit trail in DB.

PAYMENT FLOW IN DETAIL



WEBHOOK SIGNATURE VERIFICATION

```
  const crypto = require('crypto');

  function verifyRazorpayWebhook(body, signature, secret) {
    const expectedSignature = crypto
      .createHmac('sha256', secret)
      .update(JSON.stringify(body))
      .digest('hex');
    return expectedSignature === signature;
  }

  // In webhook handler:
  app.post('/webhook/payment', express.raw({ type: 'application/json' }), (req,
res) => {
    const signature = req.headers['x-razorpay-signature'];
    if (!verifyRazorpayWebhook(req.body, signature, process.env.WEBHOOK_SECRET))
{
      return res.status(400).json({ error: 'Invalid signature' });
    }
    // Process event...
  });
```

IDEMPOTENCY KEY IMPLEMENTATION

```
  // Middleware to check idempotency
  async function idempotencyMiddleware(req, res, next) {
    const key = req.headers['x-idempotency-key']
      || generateKey(req.user.id, req.body.seatId, req.body.amount);

    const stored = await redis.get(`idem:${key}`);
    if (stored) {
      return res.json(JSON.parse(stored)); // Return cached response
    }

    // Store response after handler runs
    const originalJson = res.json.bind(res);
    res.json = async (data) => {
      await redis.setex(`idem:${key}`, 86400, JSON.stringify(data));
```

```
      originalJson(data);
    };

    next();
  }

  function generateKey(userId, seatId, amount) {
    return crypto.createHash('sha256')
      .update(`${userId}:${seatId}:${amount}`)
      .digest('hex');
  }


SAGA PATTERN (Choreography)
----------------------------
  Forward transactions (happy path):
    Step 1: Lock seat in Redis + DB         → emit SeatLocked event
    Step 2: Create LOCKED booking           → emit BookingCreated event
    Step 3: Initiate payment                → emit PaymentInitiated event
    Step 4: Receive webhook confirmation    → emit PaymentConfirmed event
    Step 5: Update booking to CONFIRMED     → emit BookingConfirmed event
    Step 6: Send ticket notification        → emit TicketSent event

  Compensating transactions (failure path):
    If Step 3 fails: Delete booking → release seat lock → emit SeatReleased
    If Step 4 fails: Same as above + trigger refund if money was charged
    If Step 5 fails: retry 3 times → if still fails → alert + manual review

  Key principle: Every forward step must have a compensating step.
  The system must be able to recover from failure at any point.


================================================================================
PAGE 6 — WEEK 5: QUEUE + MICROSERVICES + gRPC (Days 27–32)
================================================================================

GOAL: Async booking confirmations working. App split into 2 services.
Services communicate via gRPC. Queue handles all async work.

-------------------------------------------------------------------------
Day | Topic                | What to build / Research first
-------------------------------------------------------------------------
27  | Queue basics         | Research: BullMQ vs RabbitMQ vs Kafka
    |                      | For this project: BullMQ (Redis-based, simple)
    |                      | Understand: producer, consumer, job, retry, DLQ
-------------------------------------------------------------------------
28  | BullMQ integration   | Add queues for:
    |                      | → booking:confirm (after payment webhook)
    |                      | → notification:send (after booking confirmed)
    |                      | → seat:release (on payment failure)
-------------------------------------------------------------------------
29  | Dead letter queue    | When a job fails 3 times → route to DLQ
    |                      | Log failure, alert via monitoring
    |                      | Manual review + retry mechanism
-------------------------------------------------------------------------
30  | Service split        | Create 2 separate Express apps:
    |                      | booking-service (port 3001): lock, state, confirm
    |                      | payment-service (port 3002): gateway, webhook
    |                      | Define clear API contracts first. Then split.
-------------------------------------------------------------------------
31  | gRPC setup           | Define booking.proto file
    |                      | Generate TypeScript/JS types
    |                      | BookingService calls PaymentService.verifyPayment
    |                      | via gRPC instead of HTTP
```

```
-------------------------------------------------------------------------
32  | API Gateway          | Single entry point (port 3000)
    |                      | Routes /bookings/* → booking-service
    |                      | Routes /payments/* → payment-service
    |                      | Auth middleware lives here
-------------------------------------------------------------------------

MILESTONE: POST /bookings triggers async job. Queue retries on fail.
2 services running independently. gRPC call works between them.


BULLMQ IMPLEMENTATION
-----------------------
  // Producer (in booking service)
  import { Queue } from 'bullmq';
  const bookingQueue = new Queue('booking-confirmation', { connection: redis });

  await bookingQueue.add('confirm', {
    bookingId,
    userId,
    seatIds,
  }, {
    attempts: 3,
    backoff: { type: 'exponential', delay: 1000 },
    removeOnComplete: true,
    removeOnFail: false,  // Keep failed jobs for debugging
  });

  // Consumer (worker process)
  import { Worker } from 'bullmq';
  const worker = new Worker('booking-confirmation', async (job) => {
    const { bookingId, userId, seatIds } = job.data;
    await confirmBooking(bookingId);
    await sendConfirmationEmail(userId, seatIds);
  }, { connection: redis });

  worker.on('failed', (job, err) => {
    console.error(`Job ${job.id} failed:`, err);
    // After max attempts → goes to DLQ automatically
  });


GRPC SETUP
-----------
  // booking.proto
  syntax = "proto3";
  service BookingService {
    rpc LockSeat (LockRequest) returns (LockResponse);
    rpc ConfirmBooking (ConfirmRequest) returns (ConfirmResponse);
  }
  message LockRequest {
    string seatId = 1;
    string userId = 2;
  }
  message LockResponse {
    bool success = 1;
    string bookingId = 2;
    string error = 3;
  }

  // gRPC server (booking service)
  import * as grpc from '@grpc/grpc-js';
  const server = new grpc.Server();
  server.addService(BookingServiceDef, {
```

```
    lockSeat: async (call, callback) => {
      try {
        const result = await lockSeatHandler(call.request);
        callback(null, { success: true, bookingId: result.id });
      } catch (err) {
        callback(null, { success: false, error: err.message });
      }
    }
  });


QUEUE VS DIRECT CALL TRADE-OFF
--------------------------------
  Direct (REST/gRPC):
    + Immediate response: know result right away
    + Simple to implement and debug
    + Good for: seat locking, payment initiation
    - Tight coupling: if downstream is down, you fail
    - No retry: failure requires client to retry

  Queue (BullMQ):
    + Decoupled: sender does not wait for receiver
    + Built-in retry: automatic backoff on failure
    + Good for: email notification, PDF generation, analytics
    - Eventual: result not immediate
    - More complex: need to poll or use websocket for status

  Rule of thumb:
    User-facing, needs immediate result → gRPC or REST
    Background work, can be async → Queue


MICROSERVICES ARCHITECTURE DIAGRAM
-------------------------------------
  [Client]
     |
     | HTTPS
     ↓
  [API Gateway :3000]
  (auth middleware, rate limit, request routing)
     |              |
     | REST         | REST
     ↓              ↓
  [Booking Svc  [Payment Svc
   :3001]         :3002]
     |    ↑ gRPC ↓    |
     |    |            |
     |    +-----------+
     |
     | BullMQ
     ↓
  [Notification Svc :3003]
  (email, SMS, push)

  All services connect to:
  [MongoDB] ← booking, seat, payment data
  [Redis]   ← locks, cache, queue, sessions


KAFKA VS BULLMQ VS RABBITMQ
-----------------------------
  BullMQ:
    + Simple, Redis-based, easy setup
    + Built-in UI (Bull Board), retry, delay, priority
```

```
    + Perfect for: job queues, scheduled tasks, this project
    - Not for: very high throughput, event log/replay

  RabbitMQ:
    + Mature, flexible routing (exchanges, bindings)
    + Good for: microservice messaging, fanout patterns
    - Needs separate broker setup, more ops overhead

  Kafka:
    + Distributed log, millions of events/sec, replay history
    + Good for: event sourcing, analytics pipeline, audit log
    - Overkill for this scale, complex to operate

  Interview answer: "For this project I chose BullMQ because we need
  job queues with retry, not a distributed event log. If we scaled to
  BookMyShow level with event sourcing and analytics, I would evaluate Kafka."


================================================================================
PAGE 7 — WEEK 6: PRODUCTION + LLD PATTERNS (Days 33–40)
================================================================================

GOAL: Production-ready code. LLD patterns applied. Can whiteboard entire
system in 10 minutes. Ready to crack machine coding round.

-------------------------------------------------------------------------
Day | Topic                | What to build / Research first
-------------------------------------------------------------------------
33  | Rate limiting        | express-rate-limit + Redis store
    |                      | Per-user: 100 req/min. Burst: 10 req/sec.
    |                      | Research: token bucket vs sliding window algo
-------------------------------------------------------------------------
34  | Caching strategy     | Write-through for show data (rarely changes)
    |                      | Cache-aside for seat availability (changes often)
    |                      | Research: cache stampede problem + solution
-------------------------------------------------------------------------
35  | Logging + tracing    | Winston structured JSON logs
    |                      | Correlation ID (UUID) per request via middleware
    |                      | Trace complete booking request across logs
-------------------------------------------------------------------------
36  | LLD design patterns  | Apply: Repository, Strategy, Observer, Factory
    |                      | Refactor existing code to use these patterns
    |                      | Write brief comment explaining why each pattern
-------------------------------------------------------------------------
37  | Load balancing       | PM2 cluster mode (use all CPU cores)
    |                      | Understand: round-robin, sticky sessions
    |                      | Research: how Redis + PM2 work together
-------------------------------------------------------------------------
38  | Circuit breaker      | Implement for payment gateway calls
    |                      | States: CLOSED → OPEN → HALF_OPEN
    |                      | Fail fast when gateway is down, fallback response
-------------------------------------------------------------------------
39  | System design        | Draw complete HLD on paper (no computer)
    | review               | Practice explaining: every component, every arrow,
    |                      | every trade-off. Time yourself: 10 minutes.
-------------------------------------------------------------------------
40  | Mock interview       | Answer all 25 questions (Page 8) without notes
    |                      | Time each answer: 2 minutes max
    |                      | Record yourself. Watch it back. Fix weak answers.
-------------------------------------------------------------------------

MILESTONE: Rate limiting live. LLD patterns applied. Can explain entire
system end-to-end fluently. Interview-ready.
```

LLD DESIGN PATTERNS APPLIED TO THIS PROJECT
---

1. REPOSITORY PATTERN
   Problem: Service layer directly calls Mongoose queries → hard to test, change DB
   Solution: Create repository layer that abstracts all DB operations

```
   // seat.repository.js
   class SeatRepository {
     async findAvailable(showId) {
       return Seat.find({ showId, status: 'AVAILABLE' });
     }
     async lockSeat(seatId, userId) {
       return Seat.findOneAndUpdate(
         { _id: seatId, status: 'AVAILABLE' },
         { $set: { status: 'LOCKED', lockedBy: userId, lockedAt: new Date() } },
         { new: true }
       );
     }
     async releaseSeat(seatId) {
       return Seat.findByIdAndUpdate(seatId, { $set: { status: 'AVAILABLE' } });
     }
   }
   // Service calls repository, not Mongoose directly
```

2. STRATEGY PATTERN
   Problem: Different locking strategies (DB lock, Redis lock, Redlock)
   Solution: Define interface, swap strategy without changing service logic

```
   class LockStrategy {
     async lock(seatId, userId) { throw new Error('Not implemented'); }
     async release(seatId, userId) { throw new Error('Not implemented'); }
   }
   class RedisLockStrategy extends LockStrategy {
     async lock(seatId, userId) { /* Redis SETNX */ }
     async release(seatId, userId) { /* Redis DEL with Lua */ }
   }
   class DBLockStrategy extends LockStrategy {
     async lock(seatId, userId) { /* MongoDB findOneAndUpdate */ }
     async release(seatId, userId) { /* MongoDB update status */ }
   }
   // BookingService receives strategy in constructor
   // Can swap locking mechanism via config without changing business logic
```

3. OBSERVER PATTERN (Event-Driven)
   Problem: After booking confirmed, need to notify, update analytics, send PDF
   Solution: Emit events, each observer handles its own concern

```
   const EventEmitter = require('events');
   const bookingEvents = new EventEmitter();

   bookingEvents.on('booking.confirmed', async ({ bookingId, userId }) => {
     await notificationService.sendConfirmation(userId, bookingId);
   });
   bookingEvents.on('booking.confirmed', async ({ bookingId }) => {
     await analyticsService.track('booking_completed', bookingId);
   });
   // In booking service:
   bookingEvents.emit('booking.confirmed', { bookingId, userId });
   // Can add more observers without changing booking logic
```

4. STATE MACHINE PATTERN

```
   Problem: Booking state transitions need to be controlled and validated
   Solution: Explicit state machine that only allows valid transitions

   const transitions = {
     INIT:            ['LOCKED'],
     LOCKED:          ['PAYMENT_PENDING', 'EXPIRED'],
     PAYMENT_PENDING: ['CONFIRMED', 'FAILED'],
     CONFIRMED:       [],  // Terminal state
     FAILED:          [],  // Terminal state
     EXPIRED:         [],  // Terminal state
   };
   async function transition(bookingId, newState) {
     const booking = await Booking.findById(bookingId);
     const allowed = transitions[booking.state];
     if (!allowed.includes(newState)) {
       throw new Error(`Invalid transition: ${booking.state} → ${newState}`);
     }
     return Booking.findByIdAndUpdate(bookingId, { state: newState, updatedAt:
new Date() });
   }

5. FACTORY PATTERN
   Problem: Different booking types (show, flight, hotel) need different logic
   Solution: Factory creates correct booking type based on input

   class BookingFactory {
     static create(type, data) {
       switch(type) {
         case 'SHOW':    return new ShowBooking(data);
         case 'FLIGHT':  return new FlightBooking(data);
         case 'HOTEL':   return new HotelBooking(data);
         default: throw new Error(`Unknown booking type: ${type}`);
       }
     }
   }

6. DECORATOR PATTERN (Middleware)
   Problem: Need to add idempotency, rate limiting, logging to handlers
   Solution: Wrap handlers with decorators, compose behavior

   const withIdempotency = (handler) => async (req, res) => {
     const key = getIdempotencyKey(req);
     const cached = await redis.get(key);
     if (cached) return res.json(JSON.parse(cached));
     // Store response after handler runs
     return handler(req, res);
   };
   const withRateLimit = (handler) => async (req, res) => {
     const allowed = await checkRateLimit(req.user.id);
     if (!allowed) return res.status(429).json({ error: 'Rate limit
exceeded' });
     return handler(req, res);
   };
   // Apply: router.post('/book', withRateLimit(withIdempotency(bookHandler)));


RATE LIMITING IMPLEMENTATION
-----------------------------
  Token bucket algorithm:
    - Each user gets N tokens per minute (e.g., 100)
    - Each request costs 1 token
    - Tokens refill at rate of N/60 per second
    - If 0 tokens → reject with 429
```

```
  Redis-based sliding window:
    const key = `ratelimit:${userId}:${Math.floor(Date.now() / 60000)}`;
    const count = await redis.incr(key);
    if (count === 1) await redis.expire(key, 60);
    if (count > 100) return res.status(429).json({ error: 'Too many
requests' });


CIRCUIT BREAKER PATTERN
------------------------
  States:
    CLOSED:    Normal operation. Requests pass through.
    OPEN:      Service is down. Fail fast, return fallback.
    HALF_OPEN: Testing recovery. Allow 1 request through.

  class CircuitBreaker {
    constructor(threshold = 5, timeout = 30000) {
      this.failures = 0;
      this.threshold = threshold;
      this.timeout = timeout;
      this.state = 'CLOSED';
      this.nextAttempt = null;
    }
    async execute(fn) {
      if (this.state === 'OPEN') {
        if (Date.now() < this.nextAttempt) throw new Error('Circuit breaker
OPEN');
        this.state = 'HALF_OPEN';
      }
      try {
        const result = await fn();
        this.onSuccess();
        return result;
      } catch (err) {
        this.onFailure();
        throw err;
      }
    }
    onSuccess() { this.failures = 0; this.state = 'CLOSED'; }
    onFailure() {
      this.failures++;
      if (this.failures >= this.threshold) {
        this.state = 'OPEN';
        this.nextAttempt = Date.now() + this.timeout;
      }
    }
  }
  // Usage: await paymentCircuitBreaker.execute(() =>
razorpay.createOrder(data));


================================================================================
PAGE 8 — KEY TRADE-OFFS (Interview Gold)
================================================================================

Know these comparisons cold. Pick a side, explain the trade-off, say when
you would choose each. Never say "it depends" without following up with
"it depends on X, if X then Y because..."

1. DB Lock vs Redis Lock
   DB lock:    Guaranteed persistent. Consistent with other DB data. Slower.
               Holds DB connection. No TTL — must clean up manually.
   Redis lock: In-memory, microsecond latency. TTL auto-release. Fast.
               Needs Redlock for multi-node safety. Extra infrastructure.
```

```
   Choose DB:  When simplicity matters more than speed. Single-server app.
   Choose Redis: Distributed system. Need TTL. High-throughput seat locking.

2. Optimistic vs Pessimistic Locking
   Optimistic:   Read freely, check on write, retry on conflict.
                 Good for: low-contention (most reads, occasional writes)
                 Bad for: high-contention (many concurrent writes on same row)
   Pessimistic:  Atomic check-and-set. Fail immediately on conflict.
                 Good for: high-contention (popular seat on ticket drop day)
                 Bad for: low-contention (wastes DB round trips on retries)

3. Saga vs Two-Phase Commit (2PC)
   2PC:  Strong consistency across services. Distributed lock held during txn.
         Single coordinator failure = system blocked. Hard to scale.
   Saga: Eventual consistency. No distributed locks. Each step local txn.
         Must write compensating transactions for every step. More code.
   Choose 2PC:  When you absolutely cannot have inconsistency. Banking core.
   Choose Saga: Distributed microservices. Can tolerate eventual consistency.

4. Queue vs Direct Service Call
   Direct (REST/gRPC): Immediate response. Tight coupling. Simpler debugging.
                       If downstream is down, caller fails immediately.
   Queue (BullMQ):     Decoupled. Retry built-in. Absorbs traffic spikes.
                       Result is async — need polling or websocket for status.
   Use direct for: Seat locking (need immediate result), payment initiation.
   Use queue for:  Email, PDF, analytics, confirmation jobs.

5. Monolith vs Microservices
   Monolith:       Simple deploy. Easy debugging. No network latency.
                   Hard to scale independently. Large codebase as team grows.
   Microservices:  Independent scaling. Team autonomy. Technology freedom.
                   Network overhead. Distributed tracing complexity. More infra.
   Right choice:   Start monolith. Split when you have a clear boundary and
                   scaling reason. Never split prematurely.

6. Cache-aside vs Write-through
   Cache-aside:    Lazy loading. Cache only when data is read.
                   Risk: cache stampede on cold start (many misses at once).
                   Good for: data that changes frequently (seat availability).
   Write-through:  Update cache on every write. Cache always warm.
                   Risk: double write on every update. Wasted if rarely read.
                   Good for: data that changes rarely (show details, venue
info).

7. gRPC vs REST
   REST:   Human-readable. Easy to debug with curl. Broad tool support.
           Text-based JSON. Slower for high-throughput.
   gRPC:   Binary protobuf. ~10x faster than REST. Typed contracts.
           Harder to debug. Needs protobuf tooling. Browser support limited.
   Use REST: Public API, client-facing, external partners.
   Use gRPC: Internal service-to-service, high-throughput, need type safety.

8. BullMQ vs Kafka
   BullMQ: Redis-based, easy setup, great UI, job queues, scheduled tasks.
           Max: millions of jobs/day. Cannot replay old jobs easily.
   Kafka:  Distributed log. Replay events. Millions of events/second.
           Complex ops. Needs Kafka cluster. Overkill for job queues.
   Choose BullMQ: Job queues, task scheduling, this booking system scale.
   Choose Kafka:  Event sourcing, audit log, analytics pipeline, massive scale.


================================================================================
PAGE 9 — 25 INTERVIEW QUESTIONS + ANSWERS
================================================================================
```

Memorize these. Practice saying them out loud. Time: 2 minutes per answer.

--- CATEGORY: CONCURRENCY ---

Q1. How do you prevent double booking when 2 users click at the same time?

    Answer:
    "The root cause is a race condition — two requests both read the seat as
    AVAILABLE before either writes the lock. The fix is to make the check-and-set
    atomic using MongoDB's findOneAndUpdate with a condition:
    findOneAndUpdate({ _id: seatId, status: 'AVAILABLE' }, { $set: { status: 'LOCKED' } })
    This is a single atomic operation. Only one request will get a non-null result.
    The other gets null — indicating the seat was already taken — and receives a 409.
    For extra safety, I also use a Redis distributed lock before hitting MongoDB,
    so the DB never sees concurrent writes on the same seat."

Q2. What is the difference between optimistic and pessimistic locking?

    Answer:
    "Optimistic locking assumes conflicts are rare. You add a version field to
    the document, read it with the current version, then update with that version
    as a condition. If another request updated first, the version changed and your
    update fails — you retry. Good for low-contention scenarios.

    Pessimistic locking assumes conflicts are likely. You use an atomic
    conditional update that only succeeds if the seat is AVAILABLE. If it fails,
    you return 409 immediately — no retry. Good for high-contention like popular
    shows on release day.

    For this project I use pessimistic at the DB level and Redis lock for the
    distributed layer."

Q3. What is a race condition? Draw the timeline that causes double booking.

    Answer:
    "A race condition is when correctness depends on the relative timing of two
    operations. In booking:
    t=0: Request A reads seat → AVAILABLE
    t=1: Request B reads seat → AVAILABLE (A hasn't written yet)
    t=2: Request A writes seat → LOCKED (succeeds)
    t=3: Request B writes seat → LOCKED (also succeeds — double booking!)
    The problem is the read and write are two separate operations with a gap
    between them. The solution is to collapse them into one atomic operation."

Q4. What is write skew? How is it different from a lost update?

    Answer:
    "Lost update: Request A and B read the same value, both compute a new value,
    B overwrites A's write. Example: both read count=5, both write count=6 instead of 7.

    Write skew: A and B read overlapping data, each makes a valid individual
    change, but the combined result violates a constraint. Classic example: on-call
    doctors — if count ≥ 2, either can go off-call. Both read count=2, both go
    off-call, now count=0 — constraint broken even though each individual

transaction looked valid.

In booking context: write skew could occur if two transactions check total seats < capacity and both approve, exceeding capacity."

--- CATEGORY: REDIS ---

Q5. Why use Redis for seat locking instead of MongoDB?

Answer:
"Three reasons: speed, TTL, and atomicity.
Speed: Redis is in-memory, O(1) operations in microseconds. MongoDB hits disk.
TTL: Redis has native key expiry. If a server crashes holding a lock, the TTL
auto-releases it — no zombie locks, no cron cleanup needed.
Atomicity: SET key value NX PX 300000 is a single atomic command —
no race between checking if lock exists and setting it.
MongoDB can do this too but it requires transactions and is slower.
Redis is purpose-built for this use case."

Q6. How does SETNX work? Why is it atomic?

Answer:
"SET key value NX PX ttl is a single Redis command where NX means 'only set if key does Not eXist' and PX is TTL in milliseconds. It's atomic because Redis is single-threaded — all commands execute sequentially with no interleaving. Two clients sending SETNX simultaneously: one executes first and gets OK, the other executes next and gets nil because the key now exists. There's no gap between the check and the set."

Q7. What happens if the server crashes while holding a Redis lock?

Answer:
"The TTL on the key ensures auto-expiry. When I set the seat lock I always include a TTL — in our case 300 seconds (5 minutes). If the server crashes, Redis continues running and when the TTL expires, the key is automatically deleted and the seat becomes available again. This is why TTL is mandatory for every lock — it's the safety net. Without TTL, a crashed server would leave seats permanently locked until manual intervention."

Q8. What is Redlock and when do you need it?

Answer:
"Single-node Redis has a failure case: if Redis restarts after a lock is acquired but before TTL expires, the lock is lost — another node can acquire it, creating two lock holders.
Redlock solves this by acquiring the lock on N/2+1 nodes out of N Redis nodes. The lock is valid only if acquired on the majority AND the total time taken is less than the TTL. Even if one node fails, the majority still holds the lock.
For this project I implement basic Redlock with 3 nodes. I'd note as a trade-off that it adds latency and complexity compared to single-node Redis."

--- CATEGORY: PAYMENT + SAGA ---

Q9. What if payment succeeds but booking confirmation fails?

Answer:
"This is handled by the Saga pattern with compensating transactions.
The webhook receives payment.captured. The booking service tries to update state to CONFIRMED. If it fails:
First, retry 3 times with exponential backoff — usually a transient error.

If all retries fail, the job goes to a dead letter queue for manual review. The payment gateway records the charge so we can trigger a refund. Critically, we never leave a user charged with no booking — the audit trail lets us reconcile and either confirm manually or refund."

Q10. A user taps Pay 3 times due to slow network. How do you prevent 3 charges?

Answer:
"Idempotency keys. Before initiating payment, I generate a key as sha256(userId + seatId + amount). On the first request, I store this key in Redis with the response (TTL 24 hours). On subsequent identical requests, I detect the key exists and return the stored response immediately — without calling the payment gateway again. The gateway never sees the duplicate requests. Only one charge is ever initiated."

Q11. What is the Saga pattern? What is choreography vs orchestration?

Answer:
"Saga is a pattern for distributed transactions where you break a long transaction into a series of local transactions, each with a compensating transaction that undoes it if something fails later.

Choreography: No central coordinator. Each service emits events, other services react. Fully decoupled but harder to trace the overall flow.

Orchestration: A central orchestrator service calls each step in sequence and handles failures. Easier to understand the flow but the orchestrator becomes a bottleneck and single point of failure.

For this project I use choreography — BookingService emits events, PaymentService and NotificationService react independently."

Q12. How do you verify a payment webhook is genuine?

Answer:
"The payment gateway signs the webhook payload with a shared secret using HMAC-SHA256. On my server I recompute the HMAC using the raw request body and my secret key, then compare it to the signature in the request header. If they match, the request is genuine. If they differ, I return 400. I also use express.raw() instead of express.json() for the webhook route because JSON parsing changes the body and breaks the HMAC check. Finally, I check the event timestamp to reject replayed old events."

--- CATEGORY: MICROSERVICES + gRPC ---

Q13. How do your services communicate? Why gRPC for service-to-service?

Answer:
"BookingService to PaymentService: gRPC. Typed protobuf contracts mean we can't accidentally send wrong data. Binary serialization is ~10x faster than JSON REST. Strong type checking catches issues at compile time.

BookingService to NotificationService: BullMQ queue. Notifications are async — the user doesn't need to wait for the email to be sent before getting a booking confirmation. Queue also handles retries automatically.

External clients: REST. Public APIs should be REST — easier to consume, browser-friendly, no special tooling needed."

Q14. What is a circuit breaker? Why do you need one?

Answer:
"When the payment gateway is down, without a circuit breaker every booking request hangs for the full HTTP timeout (say 30 seconds), exhausting your

connection pool and cascading the failure across your system.
    A circuit breaker has three states:
    CLOSED: normal, requests pass through.
    OPEN: threshold of failures hit — fail immediately, return fallback response.
    HALF_OPEN: after timeout, probe with one request — if succeeds, close circuit.
    This makes failure fast and predictable instead of slow and cascading.
    I implement it wrapping every Razorpay API call."

Q15. How do you handle service discovery?

    Answer:
    "For this project scale I use simple environment variable URLs — each service
    has the hostname of other services in its env config. Simple and works well
    at small scale.
    For production Kubernetes deployment, I'd use Kubernetes DNS — each service
    gets a DNS name (booking-service.default.svc.cluster.local) that automatically
    routes to healthy pods with built-in load balancing.
    For non-K8s production, Consul provides service registration, health checks,
    and DNS-based discovery."

Q16. Difference between synchronous and asynchronous communication?

    Answer:
    "Synchronous (REST, gRPC): Caller waits for response. Tight coupling —
    if downstream is down, caller fails. Simpler to reason about. Good when
    you need the result immediately (seat lock, payment initiation).
    Asynchronous (Queue): Caller fires and forgets. If downstream is down,
    message waits in queue. More resilient. Harder to debug. Good for
    background work (email, PDF, analytics) where the user doesn't need to
    wait for the result."

--- CATEGORY: SYSTEM DESIGN ---

Q17. How would you scale this system for 1 million users on ticket drop day?

    Answer:
    "Multiple layers:
    1. CDN: Cache show info, seat maps as static content.
    2. API Gateway: Rate limit aggressively on ticket drop — 5 req/sec per user.
    3. Horizontal scaling: Multiple Node.js instances behind load balancer.
    4. Redis cluster: Shard locks and cache across 3+ Redis nodes.
    5. Queue: BookMyShow drops to queue at DB layer — process confirmations
       async instead of making users wait.
    6. DB sharding: Shard Seat collection by showId — all seats for one show
       on same shard for locality.
    7. Read replicas: Seat availability reads hit replica, only writes hit primary.
    Bonus: Virtual queue for ticket drop — users get a position, system processes
    in order rather than accepting all concurrent requests."

Q18. How do you design the database schema?

    Answer:
    "Seat document:
    { _id, showId, row, col, status, version, lockedBy, lockedAt }
    Key decisions:
    - version field for optimistic locking
    - lockedBy + lockedAt for pessimistic tracking and cron cleanup
    - Compound index on { showId: 1, status: 1 } for fast available-seat queries

Booking document:
{ _id, userId, seatIds: [], state, idempotencyKey, paymentId, amount, createdAt }
Key decisions:
- seatIds as array (user can book multiple seats)
- idempotencyKey indexed for fast duplicate detection
- Full state history as a subdocument for audit trail"

Q19. What is eventual consistency? Where does it appear in your system?

Answer:
"Eventual consistency means the system will reach a consistent state, but not necessarily immediately after a write.
In this system:
1. Seat availability cache: Redis cache may be 60 seconds stale. User might see a seat available that was just booked. Acceptable — the lock will catch it at booking time.
2. Notification: Email is sent async after confirmation. Brief window where booking is confirmed but email not sent yet.
3. Analytics: Booking events processed async — reports may lag by minutes.
The critical path (seat locking, payment, booking state) is strongly consistent. Only peripheral systems are eventually consistent — that is the trade-off I consciously made."

Q20. What are the failure modes and how do you handle each?

Answer:
"Lock expiry: User walks away during payment. Redis TTL releases lock after 5 minutes. Cron job scans lockedAt < now-5min and resets status to AVAILABLE.

Payment fail: Saga compensation — release Redis lock, update state to FAILED,
trigger refund if partially charged, notify user.

Service crash: BullMQ jobs persist in Redis. Worker restarts and resumes processing. Incomplete bookings have TTL — expire automatically.

DB unavailable: Circuit breaker activates — fail fast with 503. Queue up retries. Return cached data where possible.

Redis unavailable: Fall back to DB-only locking. Slower but safe. Alert ops.

Duplicate requests: Idempotency keys prevent double processing.
External webhook replay: Check event timestamp, reject if older than 5 minutes."

--- CATEGORY: LLD + PATTERNS ---

Q21. Which design patterns did you apply and why?

Answer:
"Repository pattern: Abstracts all MongoDB queries behind an interface.
BookingService calls BookingRepository — easy to switch ORM or mock in tests.

Strategy pattern: LockStrategy interface with RedisLockStrategy and DBLockStrategy implementations. Swap locking mechanism via config.

Observer/Event pattern: After booking confirmed, emit event — notification observer sends email, analytics observer tracks metric. Add new behavior without touching booking logic.

State machine: Explicit valid transitions prevent invalid state changes like INIT → CONFIRMED directly.

Decorator: withIdempotency() and withRateLimit() wrap route handlers — compose cross-cutting concerns without modifying business logic."

Q22. How do you make your API idempotent?

Answer:
"Client sends an idempotency key in header (X-Idempotency-Key) or I generate one server-side as sha256(userId + seatId + amount).
Before processing, I check Redis for this key. If found, return stored response.
After processing, store the response with 24-hour TTL.
This means the client can safely retry on network failure — they'll get the same response without the operation being executed twice.
Critical for: payment initiation, booking creation, any non-GET endpoint."

Q23. How do you implement rate limiting?

Answer:
"Sliding window in Redis:
Key: ratelimit:{userId}:{window} where window = Math.floor(Date.now()/60000)
On each request: INCR the key, set TTL on first increment.
If count > 100: return 429 Too Many Requests.
For the booking API specifically: stricter limit (10 req/min per user) to prevent seat hoarding bots.
At API Gateway level: global rate limit (1000 req/sec total) as a burst protection regardless of user."

Q24. What is the Repository pattern and why use it?

Answer:
"The Repository pattern provides an abstraction layer between the business logic (services) and the data access layer (Mongoose/DB queries).
SeatRepository has methods: findAvailable(showId), lockSeat(seatId, userId), releaseSeat(seatId).
BookingService calls these methods — it has no Mongoose code.
Benefits:
1. Easy to test: inject a mock repository in unit tests.
2. Easy to change DB: swap MongoDB for Postgres without touching services.
3. Single responsibility: all query logic in one place.
It is one of the patterns that makes code genuinely maintainable."

Q25. Walk me through the complete flow when a user books a ticket.

Answer:
"1. User opens app → GET /shows → served from Redis cache (cache-aside)
2. User selects show → GET /shows/:id/seats → served from Redis cache
3. User selects seat, clicks Reserve
4. POST /bookings/lock with seatId + JWT
   → auth middleware validates JWT, extracts userId
   → rate limit check (10 req/min)
   → idempotency key check
   → Redis SETNX seat:lock:{seatId} = userId, TTL 300s
   → if Redis lock fails: return 409, seat already taken
   → MongoDB findOneAndUpdate: status AVAILABLE → LOCKED
   → create Booking record with state=LOCKED
   → return { bookingId, expiresAt }
5. POST /payment/initiate with bookingId
   → update booking state to PAYMENT_PENDING
   → create Razorpay order
   → return { orderId, amount } to client
6. Client completes Razorpay checkout

```
    7. Razorpay sends webhook to POST /webhook/payment
       → verify HMAC signature
       → if payment.captured: update booking → CONFIRMED, send job to queue
       → if payment.failed: run saga compensation
    8. BullMQ worker processes confirmation job
       → send confirmation email via notification service
       → invalidate seat cache
    Total time user waits: under 2 seconds for steps 1-6.
    Steps 7-8 are async."


================================================================================
PAGE 10 — WHAT TO FOCUS ON (Priority Matrix)
================================================================================

HIGHEST PRIORITY (Master these first — 80% of interview questions come from
here)
--------------------------------------------------------------------------------
---
  1. Race condition + how to prevent it (optimistic + pessimistic locking)
  2. Redis distributed lock with TTL (SETNX, auto-release, why Redis over DB)
  3. Payment failure handling + Saga pattern
  4. Idempotency (what it is, how to implement, where to apply)
  5. State machine design for booking lifecycle

HIGH PRIORITY (Will definitely be asked — prepare clear answers)
----------------------------------------------------------------
  6. System design HLD — draw it without notes in 10 min
  7. Trade-off questions (DB vs Redis lock, Queue vs direct, gRPC vs REST)
  8. MongoDB transactions + isolation levels
  9. Circuit breaker pattern
  10. How you'd scale to 1M users

MEDIUM PRIORITY (Good to know — may come up in deeper rounds)
-------------------------------------------------------------
  11. Redlock algorithm details
  12. Kafka vs BullMQ vs RabbitMQ decision
  13. LLD patterns (Repository, Strategy, Observer, State Machine)
  14. Cache-aside vs write-through
  15. Dead letter queue + retry strategy

KNOW THE CONCEPT (Enough to explain when asked)
-----------------------------------------------
  16. gRPC + protobuf (what, why, vs REST)
  17. Service discovery approaches
  18. Load balancing strategies
  19. Token bucket vs sliding window rate limiting
  20. Event sourcing concept (Kafka use case)


TOPICS THAT WERE MISSING FROM YOUR ORIGINAL PLAN (Added Here)
-------------------------------------------------------------
  1. JWT refresh token flow + blacklisting on logout
  2. HMAC webhook signature verification
  3. Idempotency key middleware implementation
  4. Circuit breaker pattern for external dependencies
  5. Correlation ID for request tracing across services
  6. Cache stampede problem (and solution: probabilistic early expiry or
locking)
  7. Redlock multi-node algorithm
  8. MongoDB compound indexes for query optimization
  9. LLD design patterns applied to the project
  10. Cron job for expired lock cleanup
  11. Rate limiting at API gateway level
```

12. DLQ (dead letter queue) handling
13. Input validation middleware (Joi/Zod)
14. Graceful shutdown (drain queue before process.exit)
15. Health check endpoint (/health) — required for load balancers

MACHINE CODING ROUND SPECIFIC TIPS

Time management:
- First 5 min: understand problem, ask clarifying questions
- Next 5 min: design schema + API contract on paper
- Remaining time: code, starting with the hardest/most interesting part

What interviewers look for:
- Do you handle the race condition correctly?
- Is your error handling clean?
- Are your variable/function names self-documenting?
- Do you write modular code (not one giant function)?
- Do you handle edge cases (seat already booked, user not authenticated)?

Common mistakes to avoid:
- Starting to code before understanding the requirements
- Writing everything in one file
- Forgetting error handling
- Not handling the concurrent case
- Using synchronous code where async is needed

Things to say out loud:
- "I'm using findOneAndUpdate with a condition to make this atomic"
- "I'm adding a version field here for optimistic locking"
- "This should go through a queue because it's async work"
- "I'd add an idempotency key here to handle retries"

These phrases signal senior engineering thinking.

FINAL NOTE

Bhupesh, if you complete this exactly as planned — one day at a time, research before code, notes after code — in 40 days you will be able to:

- Explain how to prevent double booking without hesitation
- Draw the entire system on a whiteboard in 10 minutes
- Answer every trade-off question with a clear reasoned opinion
- Write clean, modular, production-quality Node.js code
- Talk about concurrency, distributed systems, and reliability like a senior engineer who has shipped real systems

That is the difference between a candidate and a hire.

==========

END OF DOCUMENT

Total: 10 sections | 40-day plan | 25 interview Q&A | Complete topic coverage

==========